거리 기반 모델과 클러스터링: kNN, k-means, GMM/EM

kNN과 편향-분산 · 차원의 저주와 k 튜닝 · k-means · GMM과 EM/LDA

Machine Learning 1 · 4주차

한국과학영재학교 (KSA)

Week 4

이번 주차 개요

"당신은 가장 가까운 다섯 친구의 평균이다" — 이 한 줄이 **k-최근접이웃(kNN)**의 알고리즘 그 자체다. 거리를 직접 재는 세 주인공: kNN(지도) · k-means(비지도) · GMM/EM(확률적 일반화).

이 주차를 마치면

- kNN: k에 따른 경계 변화와 **편향-분산**
- 차원의 저주 $p^{1/n}$과 k 튜닝
- k-means가 SSE를 줄이며 수렴하는 이유
- GMM이 MLE로 안 풀리는 이유, EM 손계산
- 옌센 부등식, **BIC**로 K 선택, **LDA**

네 차시 한 줄

- **4.1** 게으른 학습 · 거리 · 편향-분산
- **4.2** 차원의 저주 · k 튜닝
- **4.3** k-means 수렴 · 엘보우 · ++
- **4.4** GMM · EM · BIC · LDA

Week 3(생성 모델)이 회피한 **차원의 저주**를 이번 장은 정면으로 맞는다. Week 5(SVM)로 가는 다리도 여기서 놓인다.

4.1 kNN: 알고리즘, 거리함수, 편향-분산 트레이드오프

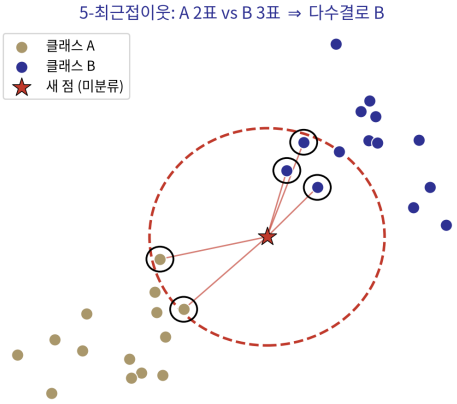
“가장 가까운 5명의 평균”

- **가장 가까운 k개의 다수결**

kNN의 성격

게으른(lazy): 예측 시점에 거리를 재는 순간 모델이 완성된다
비모수(nonparametric): 파라미터 없이 점과 점을 비교

그림으로: 가장 가까운 k개의 다수결



새 점(별)을 둘러싼 점선 원 안이 가장 가까운 $k = 5$개 — 검정 테두리로 표시된 5개 중 클래스 B가 3표로 다수, A가 2표 ⇒ 새 점은 B로 예측.

학습이 없는 학습 알고리즘

선형/로지스틱 회귀

- 데이터를 보고 파라미터 w를 최적화하는 **학습 단계**가 있음
- 학습이 끝나면 w만 있으면 원본 데이터 불필요

kNN

- "학습"은 데이터를 **그냥 저장**해 두는 것뿐
- 모든 계산은 **예측 시점**에: 새 점이 들어오면 그때서야 모든 점까지 거리를 재고 가까운 점 찾기

장점 — 언제나 최신

데이터스트림처럼 분포가 예측 직전까지 바뀌어도, 저장된 점이 곧 모델이라 **최신 데이터**로 예측. 선형회귀는 "낡은 w"를 계속 쓴다.

단점 — 예측이 m배 비쌈

예측 1번마다 m번의 거리 계산 $\rightarrow O(m \cdot d)$. 선형회귀는 $w^\top x$ 한 번($O(d)$)으로 끝남.

역사: Cover–Hart(1967)의 보증

믿기 힘든 정리

"가장 가까운 이웃의 답을 따른다"는 단순한 방법도 **데이터가 무한히 많으면** 오류율이 **베이즈 오류율**(어떤 방법으로도 줄일 수 없는 이론적 하한)의 **2배를 넘지 못한다**.

- 더 놀라운 점: **1차원**(특징 1개)에서는 2배가 아니라 **정확히 베이즈 오류율 자체**로 수렴
- 그래서 kNN은 "맹목적인 방법"이 아니라, **어디까지 틀릴 수 있는지 이론적으로 보증된 방법**



Thomas M. Cover (1938–2012) — Stanford 전기공학·통계학 교수
“Nearest Neighbor Pattern Classification” 공동저자 (Cover & Hart, 1967)

알고리즘: 가장 가까운 k개의 다수결

1. 모든 학습점 $x^{(i)}$와 x의 거리를 계산
2. 가장 가까운 k개를 고른다
3. 그 k개의 라벨 중 **다수결**(분류) 또는 **평균**(회귀)을 예측

```
def knn_predict(X_train, y_train, x_new, k):
    distances = []
    for i in range(len(X_train)):
        d = sum((X_train[i][j] - x_new[j]) ** 2
                for j in range(len(x_new))) ** 0.5
        distances.append((d, y_train[i]))
    distances.sort(key=lambda p: p[0])
    k_nearest_labels = [label for _, label in distances[:k]]
    return max(set(k_nearest_labels), key=k_nearest_labels.count)  # 다수결
```

비용은 전부 예측 시점에 몰려 있다

단계	비용	비고
거리 계산(1)	$O(m \cdot d)$	m개 점 모두 d차원 비교
정렬(2)	$O(m \log m)$	사실상 가장 비싼 작업
다수결(3)	$O(k)$	무시할 만

- 100만 개로 학습한 kNN은 예측 **한 번**에 100만 번 거리 계산

실용적 해결

KD-트리, **ANN**(approximate nearest neighbor, 예: HNSW, Malkov & Yashunin 2016) 같은 인덱스 자료구조로 "거리를 다 재지 않고도 가까운 점만 찾는다". 개념적으로는 항상 "모든 점을 다 본다"는 전제로 이해.

회귀용 kNN과 가중 평균

분류의 “다수결”을 **가장 가까운 k개의 y값 평균**으로 바꾸면 바로 **회귀** 알고리즘:

```
def knn_predict_regression(X_train, y_train, x_new, k):
    d = [sum((a - b) ** 2 for a, b in zip(xi, x_new)) ** 0.5
         for xi in X_train]
    k_idx = sorted(range(len(X_train)), key=lambda i: d[i])[:k]
    return sum(y_train[i] for i in k_idx) / k
```

거리 가중 평균

$$\hat{y}(x) = \frac{\sum_{i \in \mathrm{NN}_k(x)} w_i \, y^{(i)}}{\sum_i w_i}, \quad w_i = \frac{1}{d(x, x^{(i)}) + \epsilon}$$

이웃이 가까울수록 그 의견의 비중이 커진다 — “가장 가까운 1개에 거의 100%”($k = 1$)와 “모든 이웃을 동등하게 본다”(평범한 평균) 사이의 연속적인 선택지.

거리 함수: p-노름과 순위 뒤집기

- **유클리드**: $d(x, x') = \sqrt{\sum_{j=1}^{n} (x_j - x'_j)^2}$
- **맨해튼**: $\sum_j |x_j - x'_j|$ (좌표축을 따라서만 이동)
- **p-노름:** $p = 2$ **유클리드,** $p = 1$ **맨해튼**

$$d_1(A, x) = 11,\ d_1(B, x) = 12 \Rightarrow p{=}1\text{에서는 } A\text{가 더 가까움}$$

$$d_2(A, x) = \sqrt{101} \approx 10.05,\ d_2(B, x) = \sqrt{72} \approx 8.49 \Rightarrow p{=}2\text{에서는 } B$$

p 키우면 거리 순위 뒤집힘

정규화: 각 특징의 "동등한 목소리"

스케일이 큰 특징이 거리를 독차지한다

MinMax 정규화

- 범위를 $0 \sim 1$로 스케일 변환
- 극단값(outlier)에 휘어질 수 있음

표준화 (z-score)

- $z_j = (x_j - \mu_j)/\sigma_j$ (평균 0, 표준편차 1)
- outlier가 있으면 조금 더 방어적

핵심 원리(둘 다)

각 특징이 거리에 "**동등한 목소리**"를 가지게 한다. 정규화가 해결된 뒤에는 p의 선택이 비로소 의미가 있다 — 실전에서 p를 1과 2 사이에서 고르는 것보다 **정규화 빼먹는 실수**가 훨씬 큰 성능 차이를 만든다.

k 트레이드오프: 편향과 분산

k 너무 작음 (예: $k = 1$)

- 노이즈 한 점에도 민감하게 반응
- 훈련 데이터를 살짝만 바꿔도 예측이 크게 요동
- **분산(variance)이 크고** $\Rightarrow$ **과적합**

k 너무 큼 (예: $k = m$, 전체)

- 항상 전체 다수결만 $\rightarrow$ 지역적 패턴 완전 무시
- 분산은 작지만 구조적으로 패턴을 못 잡음
- **편향(bias)이 크고** $\Rightarrow$ **과소적합**

이 관계는 kNN만의 얘기가 아니라 **지도학습 전체**에 적용되는 원리. 회귀의 기대 오차는 세 항으로 분해된다:

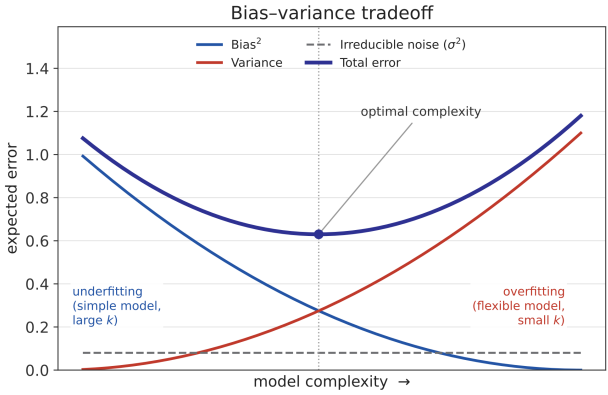
편향-분산 분해와 수치 실험

$$\mathbb{E}\left[(y - \hat{f}(x))^2\right] = \underbrace{\left(\text{Bias}[\hat{f}(x)]\right)^2}_{\text{구조적으로 못 맞추는 정도}} + \underbrace{\text{Var}[\hat{f}(x)]}_{\text{표본이 바뀔 때 예측이 흔들림}} + \underbrace{\sigma^2}_{\text{데이터 잡음(줄일 수 없음)}}$$

k	Bias^2	Var
1	0.001	0.255
5	0.006	0.056
50	0.470	0.019

$k = 1$**: 분산 0.255,** $k = 50$**: 편향 0.470,** $k = 5$**: 중간**

편향-분산 트레이드오프



- **너무 단순**(k 큰 kNN, 얕은 트리, 규제가 강한 선형모델) → Bias ↑, Variance ↓ = **과소적합**
- **너무 유연**($k=1$ kNN, 아주 깊은 트리, 대형 신경망) → Variance ↑, Bias ↓ = **과적합**
- 최적 모델 복잡도 = 둘의 합(전체 오차)이 **최소** 되는 지점
- 이 프레임은 Week 6(정규화), Week 7(트리 가지치기), Week 9(신경망 정규화)에서 "과적합을 어떻게 막나"를 다룰 때마다 다시 등장

“분산이 크다”의 구체적 그림

$k = 1$

- 잘못 찍힌 점 바로 옆에서 “그 점이 정답”
- **경계 전체가 한 점의 위치를 따라 출렁**

$k = 5$

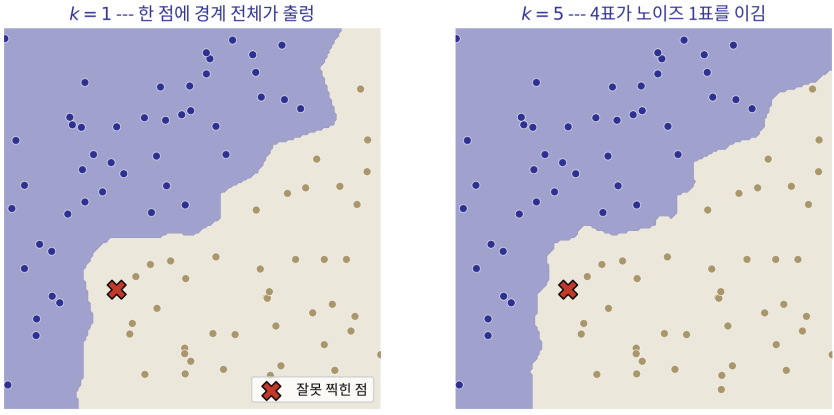
- 잘못 찍힌 한 점을 4개 정상점이 **투표로 이김**
- 그 한 점은 경계에 거의 영향 없음

복잡한 경계는 $k = 5$로 매끄럽게 그어짐

정의의 kNN 번역

분산이 크다 = **훈련 데이터의 미세한 변화**(노이즈 한 점, 표본 재추출)가 **예측의 미세한 변화**(경계의 출렁임)를 낳는 정도. kNN에서는 이 인과가 **눈으로 직접 확인** 가능 — 신경망에서 “분산”을 눈으로 보기 훨씬 어렵다.

그림으로: 잘못 찍힌 한 점과 k



같은 데이터, 경계 근처 점(X) 하나만 라벨이 뒤집혔다 — $k = 1$은 그 점 주변 경계 전체가 출렁이지만, $k = 5$는 나머지 4표가 눌러 거의 영향이 없다.

손으로 한 번: kNN 예측 직접 계산

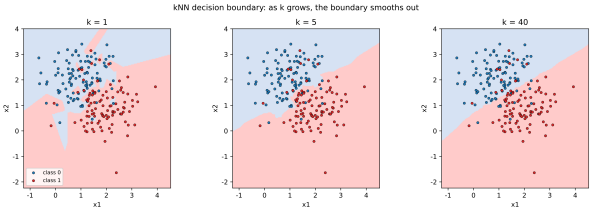
$$(1,1) \to 0,\ (2,1) \to 0,\ (1,2) \to 0,\ (8,8) \to 1,\ (9,8) \to 1,\ (8,9) \to 1$$

새 점 $(7,7)$을 $k = 3$으로 분류 — 유클리드 거리 계산:

학습 데이터	라벨	$(7,7)$까지 거리
(8,8)	1	$\sqrt{2} \approx 1.414$
(9,8)	1	$\sqrt{5} \approx 2.236$
(8,9)	1	$\sqrt{5} \approx 2.236$
(2,1)	0	$\sqrt{61} \approx 7.810$
(1,2)	0	$\sqrt{61} \approx 7.810$
(1,1)	0	$\sqrt{72} \approx 8.485$

가장 가까운 3개 — (8,8), (9,8), (8,9) — 가 **모두 라벨 1** $\Rightarrow$ 다수결로 $(7,7)$은 **1(합격)**.

결정경계: k를 키우면 어떻게 변하나



- 두 원형 blob(클래스 0: 왼쪽 위, 1: 오른쪽 아래) 200개 점, $k = 1, 5, 40$의 결정경계
- $k = 1$: 훈련점 주변에 **톱니바퀴**처럼 복잡하게 접힘 — 각 점의 "영향권"(보로노이 셀) 경계가 그대로 결정경계
- $k = 5$: 톱니가 눈에 띄게 **매끄러워짐** (한두 점의 노이즈는 5표에서 다수결을 못 뒤집음)
- $k = 40$(훈련점 140개 중 1/3): **거의 직선** $\rightarrow$ 지역 패턴 무시, 과소적합의 공간적 모습

매끄러움은 검증 정확도로 판단

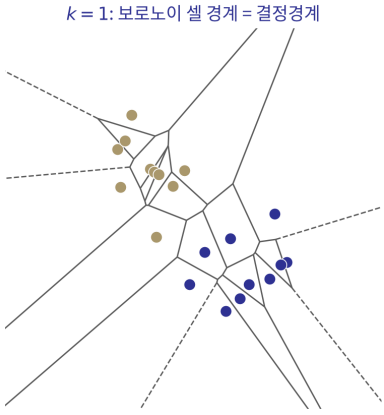
기하학적 성격: 보로노이 다이어그램

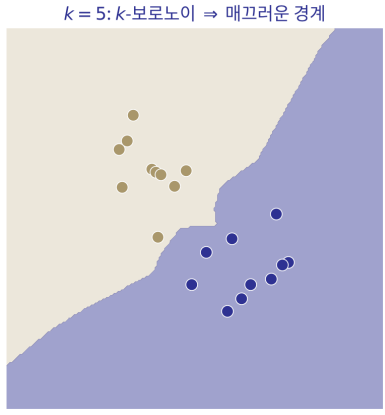
- $k = 1$ **kNN의 결정경계 = 보로노이 다이어그램의 경계** (Voroni, 1908)과 정확히 일치
- **보로노이 셀: 볼록 다각형, 수직등분선 경계**

두 가지 실용적 의미

1. $k = 1$ 경계는 **항시 각진(비선형) 다각형의 합** — 아무리 많은 점을 써도 **곡선을 그을 수 없다**(충분히 작은 직선 조각으로만 근사)
2. **두 훈련점이 아주 가까우면** 그 수직등분선이 두 점 사이를 지나, 근처 경계가 두 점의 **상대적 위치 에만** 달려 있다 = 분산 얘기의 기하학적 번역

그림으로: 보로노이 다이어그램





왼쪽: $k = 1$의 결정경계는 각 훈련점을 중심으로 한 보로노이 셀의 경계 그 자체(각진 다각형). 오른쪽: $k = 5$는 여러 셀의 다수결로 경계가 매끄러워진다.

자주 하는 실수: 스케일이 다른 특징을 그대로 섞기

공부 시간과 가구 소득으로 합격 예측

$$d(P_1, \text{new}) \approx 1000.0005, \qquad d(P_2, \text{new}) \approx 950.026$$

0 1 정규화 적용

$$d(P_1, \text{new}) \approx 1.008, \qquad d(P_2, \text{new}) \approx 1.292$$

정규화의 효과, 숫자로: 0.556 $\rightarrow$ 1.000

x 범위 0.6 vs y 범위 9

	검증 정확도 ($k = 3$)
정규화 **없이**	0.556
정규화 **후**	**1.000**

해석

"y 잡음으로만 분류하던 모델"이 정규화 후 x가 분리 축으로 정상 작동 $\rightarrow$ 정확도 **거의 두 배**. 정규화 없이 성능이 40%나 깎일 수 있는 **극단적 사례** — 센서·금융 데이터 등 실데이터에서 이런 스케일 차이는 흔하다.

베이스라인으로서의 kNN

- **kNN은 느리지만 베이스라인으로 쓰임**

정석

"새 데이터셋을 받았다, 어떤 모델을 써볼까?" $\rightarrow$ **먼저 kNN으로 베이스라인을 잡아봐라**.

- kNN은 가정이 거의 없으므로 데이터의 **"학습 가능성" 하한**을 알려줌
- kNN으로조차 좋은 성능이 안 나오면 더 복잡한 모델도 (거의) 확실히 어려움 = **조기 경보기**
- kNN으로 90%를 나갔다면, 더 복잡한 모델은 **그 90%를 초과해야** 의미가 있음

자주 묻는 질문 (4.1)

- **Q. 왜 "학습이 없다"고 하나요?** 선형회귀는 데이터를 w 몇 개로 **압축**해 저장, kNN은 **원본 전체**를 갖고 있어야 예측 가능 — 파라미터 최적화 과정 자체가 없음
- **Q. k는 항상 홀수?** 이진 분류에선 동점(2:2) 방지를 위해 홀수가 관례적. 클래스 ≥ 3개면 홀수여도 동점이 날 수 있어 **동점 처리 규칙**을 미리 정해두는 것이 안전
- **Q. kNN으로 회귀를 해도 되나요?** 된다 — "가장 가까운 k개 y의 평균" = **국소적 평균 smoother**, 비모수 회귀의 가장 기본적인 형태(신경망이 "복잡한 비모수 근사"라면 kNN 회귀는 "가장 단순한 비모수 근사"). 단, 차원이 높아지면(4.2) 이웃 평균이 노이즈에 완전히 지배됨
- **Q. 예측 확률 $P(y=1|x)$는?** 가장 가까운 k개 중 **클래스 1의 비율** $\hat{P}(y=1|x) = \frac{\#\{\text{클래스 1 이웃}\}}{k}$ — 다수결 분류기의 확률적 버전(0.5 초과 시 1로 예측). k가 클수록 안정적, 실전에선 거리 가중 ($w_i = 1/d^2$ 등)으로 고치기도
- **Q. 특징 100개 넘으면?** 쓸 수는 있으나 **성능이 크게 떨어짐**(차원의 저주). 실전 고차원 kNN = **특징 선택** 또는 **차원 축소**(Week 14 PCA)를 먼저 — 표준적 절차. 특징 10개 미만의 저차원 데이터라면 여전히 경쟁력

실습으로 연결하기 (4.1)

노트북 `chapter04_1_knn.ipynb`에서:

1. 두 blob 데이터에 kNN을 적용, $k = 1, 5, 40$**의 결정경계를 나란히** 그림
2. 같은 데이터에 k를 1$\rightarrow$40까지 쭉 늘리며 **훈련 정확도 vs 검증 정확도** 그래프 — "과적합 $\rightarrow$ 정상 $\rightarrow$ 과소적합" 궤적을 직접 관찰
3. 1차원 sin 함수 + 잡음 데이터에서 **편향·분산을 수치적으로 측정** (앞 표의 숫자가 여기서 나옴)
4. 스케일이 다른 특징 데이터를 **정규화 전/후**로 나누어 kNN 정확도 비교 (0.556 vs 1.000 예시가 여기서 나옴)

4.2 차원의 저주와 실습: 검증셋으로 k 튜닝하기

차원의 저주(Curse of Dimensionality): “가까운 이웃”이 멀어진다

특징이 늘어날수록, “가장 가까운 이웃”조차 점점 멀어진다.

- **부피는 0.5^n으로 기하급수적으로 증발**

n차원 단위 정육면체 $[0,1]^n$에 점이 균등 분포, 전체의 비율 p를 포함하려면 한 변 ℓ:

$$\ell^n = p \quad \Longrightarrow \quad \ell = p^{1/n}$$

$p = 0.01$**(전체의 1%)일 때의** ℓ**:**

n	$\ell = p^{1/n}$	
1	0.01	
10	0.63	
100	0.955	**각 변 길이의 95.5%나** 차지해야 겨우 1%

동거리 현상: 순서가 무너진다

- **이웃들이 "모두 비슷한 거리"에 놓이는 현상** (거리 집중, distance concentration)

n	1번째 이웃	11번째 이웃	비율(11/1)
2	0.033	0.142	**4.33**
5	0.253	0.514	2.04
10	0.617	0.894	1.45
20	1.151	1.429	1.24
50	2.246	2.509	**1.12**

차원이 높아지면 거리 차별화 불가

동거리의 수학적 근거: $k^{1/n}$ 스케일

n차원 구 반지름 $f^{1/n}$ 스케일

n	이론 $11^{1/n}$	시뮬레이션
2	3.32	4.33
10	1.27	1.45
50	1.05	1.12

실험: 노이즈 특징을 붙여가면?

잡음 특징 수 (총 차원)	검증 정확도 ($k = 7$)
0 (2차원)	**0.867**
5 (7차원)	0.813
20 (22차원)	0.680
50 (52차원)	**0.507**

해석

52차원이면 **0.507** — 동전 던지기(0.500)와 사실상 무차이. 원래 들어 있던 **유용한 2차원 신호는 그대로**인데, **아무것도 알려주지 않는 축 50개**를 추가했을 뿐 $\Rightarrow$ 순수하게 차원의 저주만으로 정확도가 **무너짐**. "특징을 더 많이 모은 것"이 오히려 **해로운 축**을 추가한 셈.

차원의 저주는 "차원의 숫자"가 아니라, 거리에 노이즈 축이 얼마나 섞여 있는가에 달려 있다.

역사: 용어의 기원 (Bellman 1957)

차원의 저주 용어 기원

이력이 주는 교훈

"왜 느려지는가"(계산 복잡도)에서 "왜 정확도가 무너지는가" (데이터 희소성)로 의미가 넓혀진 이 사실은, 우리가 다루는 것이 본질적으로 **기하학적/데이터적 문제**(인접한 점들이 동거리에 놓임)라는 점을 상기시켜 준다 — 계산 효율의 문제가 아니라, **공간 자체가** n차원이 되면서 변하는 성질.



Richard Bellman (1920–1984)
1957년경 동적계획법(dynamic programming) 연구에서 “차원의 저주”라는 용어를 처음 사용

자주 묻는 질문 (4.2-a)

- **Q. 차원의 저주는 kNN에만 해당하나요?** 아니다 — **거리/유사도를 직접 재는 모든 방법**(k-means, 커널 SVM, PCA 이전 원본 공간의 유사도 등)이 정도만 다를 뿐 같은 문제. 반면 **트리 기반**은 한 번에 특징 하나씩만 보고 나누므로 상대적으로 덜 취약. “이 모델이 거리를 직접 쓰는가?”가 취약도를 가늠
- **Q. 차원이 많으면 항상 특징을 줄여야 하나? 꼭 그렇지는 않다** — 유용한 정보가 소수 축에 몰려 있으면 **특징 선택/차원 축소**(PCA)가 도움. 반대로 각각이 **독립적으로 유의미한 신호**를 담고 있으면(Week 3 나이브베이즈가 바로 이런 설계) 무작정 줄이는 것이 오히려 정보 손실

자주 묻는 질문 (4.2-b): 속도/메모리는 별개의 문제

- **Q. 고차원에서는 "속도"도 문제되지 않나요?** 된다 — 그리고 이는 정확도 문제(동거리)와 **서로 다른** 문제. 순진한 구현은 예측 1번에 $O(m \cdot n)$ 연산: $m = 10^6, n = 100$이면 약 10^8 회 곱셈/덧셈 — numpy로 0.1초 이내, 순수 Python 루프로는 수 초
- 더 큰 문제는 **메모리**: 10^6개 점의 좌표를 double(8바이트)로 저장 = 8×10^8 바이트 $\approx$ **800MB** + 예측 시점의 중간 배열
- 실전에서는 **k-d 트리**(Friedman, 1970) 같은 탐색 가속 자료구조를 쓰지만, 대략 $n > 20$ 이상에서는 탐색 효율이 급격히 떨어져 **선형 탐색과 비슷**해짐

정리

"고차원에서는 kNN이 느리다" = **정확도** 문제와 **속도/메모리** 문제가 **독립적으로** 모두 성립하는 두 가지의 합.

실습: 검증셋으로 k 튜닝

4.1절에서 "적절한 k는 검증 데이터로 정한다"고 했다 — 직접 해보자.

1. 데이터를 **훈련/검증**으로 분리
2. 여러 k 각각에 대해 **검증셋 정확도** 계산
3. 가장 높은 지점을 고른다

`make_moons`(scikit-learn) 500개 점, 30% 검증 / 70% 훈련, $k \in \{1, 3, 5, 7, 10, 15, 20, 30, 50\}$:

k	훈련 정확도	검증 정확도
1	**1.000**	0.900
5	0.940	0.920
7	0.931	**0.960**
15	0.934	0.947
30	0.929	0.947
50	0.920	0.933

$k = 1$: 훈련 100%에 가깝지만(과적합) 검증은 오히려 떨어짐 — 편향-분산 트레이드오프가 숫자로 드러나는 지점. 검증 정확도는 k가 작을 때(분산)와 클 때(편향) **양쪽 끝에서 모두 낮아지는 "U" 모양**. 이 표에선 $k = 7$(0.960)이 최고.

tune_k 코드

```python
def train_val_split(X, y, val_ratio=0.3, seed=0):
    import random
    idx = list(range(len(X)))
    random.Random(seed).shuffle(idx)
    n_val = int(len(X) * val_ratio)
    val_idx, train_idx = idx[:n_val], idx[n_val:]
    return ([X[i] for i in train_idx], [y[i] for i in train_idx],
            [X[i] for i in val_idx], [y[i] for i in val_idx])

def tune_k(X, y, k_candidates):
    X_train, y_train, X_val, y_val = train_val_split(X, y)
    best_k, best_acc = None, -1.0
    for k in k_candidates:
        correct = sum(knn_predict(X_train, y_train, xv, k) == yt
                      for xv, yt in zip(X_val, y_val))
        acc = correct / len(X_val)
        if acc > best_acc:
            best_k, best_acc = k, acc
    return best_k
```

자주 하는 실수: 훈련 정확도로 k 고르기

실수: 훈련 데이터 정확도로 k 고르기

```
def tune_k_wrong(X, y, k_candidates):
    X_train, y_train, X_val, y_val = train_val_split(X, y)
    best_k, best_acc = None, -1.0
    for k in k_candidates:
        acc = sum(knn_predict(X_train, y_train, xt, k) == yt
                  for xt, yt in zip(X_train, y_train)) / len(X_train)
        if acc > best_acc:
            best_k, best_acc = k, acc
    return best_k   # always returns 1
```

훈련 정확도 최대화 k는 구조적으로 $k = 1$

확인 문제: ℓ를 직접 계산

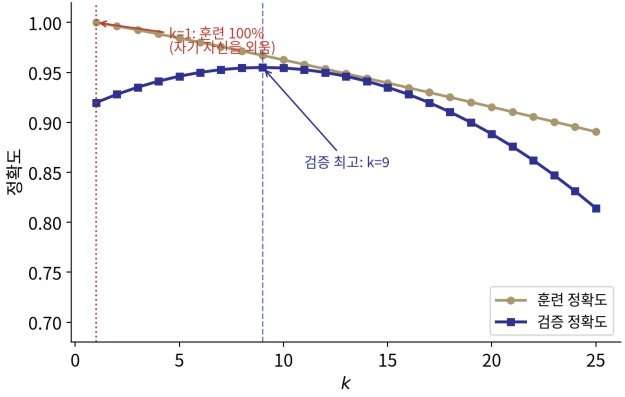
앞 표에선 $n = 1, 10, 100$만 계산했다 — $n = 20$**일 때** $\ell = 0.01^{1/20}$**은?**

- 계산기 전에 추정해보자: $n = 10$일 때 0.63, $n = 100$일 때 0.955였으니 $n = 20$은 그 사이 어디쯤
- **힌트**: ℓ가 n에 대해 **선형**으로 늘어나는 게 아니라, n이 작을 때 **훨씬 빠르게** 커진 뒤 1에 가까워지며 증가폭이 줄어듦에 주의
- 직접 `0.01 ** (1/20)`를 계산해 예상과 비교 (참고값: ≈ 0.794)

이 숫자의 뜻

특징이 20개만 되어도 "근처 1%"를 찾으려면 각 축의 **대부분**(약 79%의 비율)을 뒤져야 한다 — 겨우 20차원에서도 "가까운 이웃"이라는 말이 **이미 상당히 무뎌지기** 시작한다.

자주 하는 실수: 검증셋 없이 훈련 정확도로 k 고르기



- $k = 1$ **시 훈련 정확도 100%**
- `tune_k`가 `X_val, y_val`(학습에 안 쓴 데이터)로 정확도를 재는 이유가 정확히 이것을 막기 위함

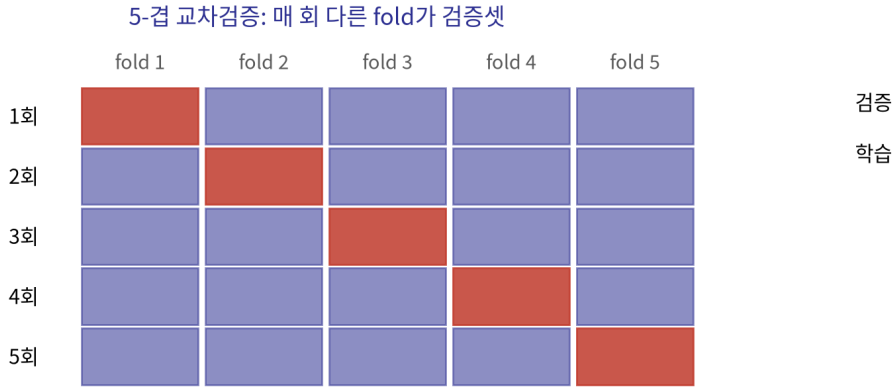
더 안정적으로: k-겹 교차검증

- **k-겹 교차검증:** **k번 반복해 성능 평균**
- 단일 분할보다 **운에 덜 휘둘리는** 추정치가 나온다.

Week 6에서 엄밀하게

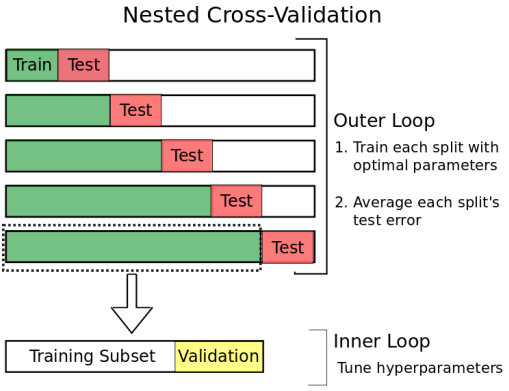
지금은 아이디어만 — k-fold 코드, 손계산 예제, CV 추정값 자체의 편향까지는 **Week 6.2**에서 λ(정규화 강도) 선택에 적용하며 제대로 다룬다.

그림으로: k-겹 교차검증



데이터를 5개 fold로 나눈 뒤, 매 회 다른 fold 하나를 검증셋(빨강)으로 남기고 나머지(보라)로 학습 — 5번 반복해 5개 정확도를 평균낸다.

cf. 시계열이면 무작위로 쪼개면 안 된다



- 일반 k-fold는 **무작위로 섞어서** 쪼갠다 — 시계열에서는 **미래 데이터가 과거 학습에 새는** 것(data leakage)과 같다.
- **Rolling-origin**(확장 윈도우): 항상 **앞 구간만 train**, 그 다음 구간을 test로 — 시간이 지날수록 train 구간이 넓어지며 반복.
- 그림의 **Outer loop**는 이렇게 test 오차를 여러 번 측정해 평균내고, **Inner loop**는 그 안에서(train 구간만으로) 하이퍼파라미터를 튜닝 — **중첩(nested) CV**.

유명한 사례: MNIST의 첫 베이스라인

- **kNN은 약 97% 정확도 기록**

실용적 교훈

새 프로젝트를 시작할 때 **항상 복잡한 모델부터** 시도할 필요는 없다 — 가장 단순한 방법(kNN, 심지어 평균/다수결)으로 **먼저 베이스라인**을 잡아두면, 이후 더 복잡한 모델이 **정말로 그만한 값어치**를 하는지 비교할 기준이 생긴다.



MNIST 손글씨 숫자 데이터셋 샘플 (LeCun et al., 1998)
1998년 발표 당시 kNN 베이스라인이 약 97% 정확도를 기록

차원이 많으면 항상 특징을 줄여야 하나?

유용한 신호가 소수 축에 몰려 있으면

- 나머지 축이 노이즈에 가깝다면
- **특징 선택** 또는 **차원 축소** (Week 14 PCA)로 의미 있는 축만 남기는 것이 도움
- = 앞 실험(50개 junk 축)의 정반대 상황

각 축이 독립적으로 유의미한 신호를 담으면

- 축이 많아도 무작정 줄이면 **정보 손실**
- Week 3 **나이브베이즈**가 바로 이런 상황을 다루는 설계 — “특징 독립” 가정으로 차원의 저주를 **피해**내는 모델

차원의 저주는 “차원의 숫자”가 아니라, 거리에 노이즈 축이 얼마나 섞여 있는가.

차원의 저주를 피하는 실전 해법 3가지

1. **특징 선택 / 차원 축소**
2. **모델 교체**
3. **유효 차원 낮추는 전처리**

어떤 해법이 맞는가

데이터에 따라 다르므로, Week 6의 **교차검증**으로 각 선택의 검증 정확도를 **직접 비교**해보는 것이 가장 안전하다.

실습으로 연결하기 (4.2)

노트북 `chapter04_2_curse_dim.ipynb`에서:

1. **동거리 시뮬레이션**(앞 표): 200개 점 균등 분포, 20개 임의 질의점에서 1번째/11번째 이웃의 평균 거리를 $n = 2, 5, 10, 20, 50$로 실측
2. **노이즈 특징 실험**(앞 표): moons 데이터에 junk 축을 0→50개 붙여가며 $k = 7$ 검증 정확도 0.867 → 0.507 추적
3. k **튜닝 실습**: `tune_k`로 $k \in \{1, 3, 5, 7, 10, 15, 20, 30, 50\}$의 검증 정확도를 계산, 최적 $k = 7$(0.960) 확인 + `tune_k_wrong`이 항상 $k = 1$을 반환하는 함정 재현

4.3 k-means 클러스터링

비지도 클러스터링: 라벨 없이 거리만으로

kNN(지도, 게으름)

- **라벨 있는** 학습점과 새 점의 거리만 예측 시점에 재음
- "새 점의 클래스 = 가까운 점의 다수결"

k-means(비지도)

- **라벨이 전혀 없는** 점들끼리
- "서로 가까운 점은 한 덩어리"라는 거리 정보 **만으로** 데이터의 구조를 쪼갬
- k개의 그룹, 각 그룹은 **중심점**(centroid)으로 대표

이 장의 축

라벨이 없다는 점에서 4.4절 **GMM/EM**(k-means의 확률적 일반화)과 직접 이어지지만, 핵심 동작(가장 가까운 대상을 찾는다)은 이번 장의 주제 그대로.

k-means 알고리즘: 할당 $\rightarrow$ 갱신 반복

1. k개의 중심점을 **무작위로 초기화**
2. **할당 단계**: 각 점을 **가장 가까운 중심점**의 그룹으로 배정
3. **갱신 단계**: 각 그룹의 새 중심점 = 그 그룹 점들의 **평균**
4. 2$\sim$3을 **할당이 더 이상 바뀌지 않을 때까지** 반복

kmeans 코드

```python
def kmeans(X, k, max_iters=100):
    import random
    centroids = random.sample(X, k)
    for _ in range(max_iters):
        clusters = [[] for _ in range(k)]
        for x in X:
            distances = [sum((x[j]-c[j])**2 for j in range(len(x)))
                         for c in centroids]
            closest = distances.index(min(distances))
            clusters[closest].append(x)
        new_centroids = [
            [sum(pt[j] for pt in cluster) / len(cluster)
             for j in range(len(X[0]))] if cluster else centroids[i]
            for i, cluster in enumerate(clusters)]
        if new_centroids == centroids:
            break
        centroids = new_centroids
    return centroids, clusters
```

이 알고리즘이 뭘 최소화하는가: SSE

목표 함수: SSE

$$J = \sum_{i=1}^{k} \sum_{x \in C_i} \|x - \mu_i\|_2^2$$

- C_i = i번 클러스터의 점 집합, μ_i = 그 중심점
- k-means = 이 J를 작게 만드는 **중심점과 분할**을 찾는 알고리즘

할당 단계가 J를 줄이는 이유

- J는 각 점의 기여 $\|x - \mu_{\text{own}}\|^2$의 합
- $\Rightarrow$ 전체 합 J는 **줄거나 그대로**

핵심

"모든 점을 **가장 가까운** 중심점에게 배정한다"는 규칙은, 정확히 **이 기여를 각 점마다 최소화**하는 동작이다.

갱신 단계가 J를 줄이는 이유

한 클러스터의 점 $\{x_1, \ldots, x_m\}$이 고정되어 있을 때, 중심점 c에 대한

$$f(c) = \sum_j \|x_j - c\|^2$$

를 미분:

$$\nabla_c f(c) = -2 \sum_j (x_j - c) \quad = 0 \iff c = \text{점들의 평균}$$

- 그래디언트가 **평균**일 때 정확히 0
- f는 c에 대해 **볼록한 이차함수** $\Rightarrow$ 이 안장점이 곧 **전역 최소점**

결론

"새 중심점 = **그룹의 평균**"은 해당 클러스터의 기여를 최소화하는 **수학적으로 유일한 선택**.

수렴 보장 — 그리고 그 한계

- 두 단계가 **각각** J를 줄이므로, J는 매 반복마다 **절대 늘어나지 않음**(단조감소)
- $J \geq 0$이고 점의 위치가 유한하므로 $\Rightarrow$ 언젠가 **변화가 멈출 수밖에** 없다 = **수렴 보장**의 전부

실습: 클러스터링 결과 시각화

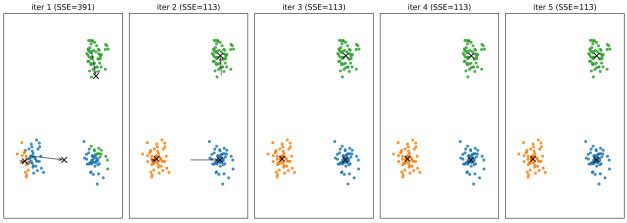
2차원 데이터라면 할당 결과를 **눈으로** 바로 확인할 수 있다 — 알고리즘이 “올바른” 그룹을 찾았는지 감으로 검증하는 가장 빠른 방법.

```
import matplotlib.pyplot as plt

def plot_clusters(clusters, centroids):
    colors = ["tab:blue", "tab:orange", "tab:green", "tab:red"]
    for i, cluster in enumerate(clusters):
        xs = [pt[0] for pt in cluster]
        ys = [pt[1] for pt in cluster]
        plt.scatter(xs, ys, c=colors[i % len(colors)],
                    label=f"cluster {i}")
    plt.scatter([c[0] for c in centroids], [c[1] for c in centroids],
                c="black", marker="x", s=100, label="centroid")
    plt.legend()
    plt.savefig("kmeans_result.png")
```

같은 데이터에 k=2, k=3, k=4를 돌려 나란히 비교하면, “몇 개의 덩어리로 나누는 게 자연스러운가”에 대한 감을 **숫자(SSE)보다 훨씬 빠르게** 얻을 수 있다.

수렴 과정 시각화



- 150점 2차원 장난감 데이터(3개 원형 덩어리, 각 50점)
- 점의 색 = 할당된 클러스터, 검은 X = 그 회차의 **중심점**, 화살표 = 이전→이후 **중심점 이동**
- 반복이 진행될수록 **화살표가 짧아지고** 마지막 회차에는 **사라짐**

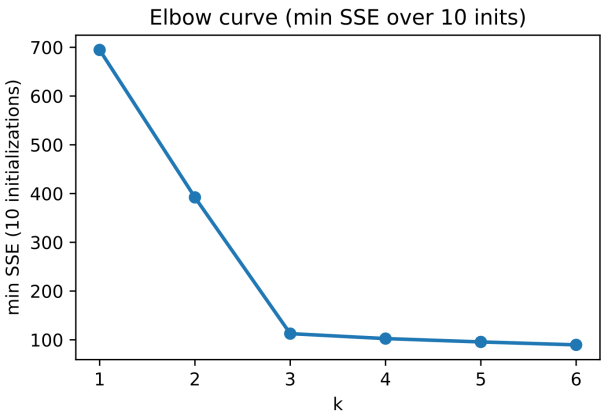
"중심점이 **그룹의 평균**으로 재계산된다"는 문장을 눈으로 확인할 수 있는 가장 빠른 방법.

k를 고르는 법: 엘보우(elbow) 방법

엘보우 방법: 감소폭 급격히 줄어드는 지점

k	1	2	3	4	
SSE(최소)	3753.3	1867.1	**258.4**	223.4	19

엘보우 곡선 읽기



- 그래프를 그리면 $k = 3$에서 **각도가 꺾이는** "팔꿈치"가 정확히 여기에 옴
- **솔직히**: 엘보우는 항상 이렇게 선명한 각도를 만들지 않음 — 덩어리가 서로 가깝거나 크기가 비슷하면 곡선이 완만해져 "여기다!"를 확신하기 어려움

완만할 때의 대응:

- (i) 여러 초기화로 SSE의 **분포** 보기
- (ii) **실루엣(silhouette) 점수** 등 다른 지표로 교차 검증

절대 하지 말 것: 최소 SSE로 k 고르기

SSE는 k가 커질수록 항상 줄어든다

원리

기준은 항상 **"감소폭이 급격히 줄어드는 지점"**이다.

- 엘보우 = **눈**으로 그 꺾임을 찾는 방법
- 4.4절 **BIC** = 같은 결정을 **하나의 숫자**로 내리는 **원리 있는 버전**
- 이 함정은 4.4절에서 "likelihood로 K를 고르면 $K = m$이 이긴다"는 모습으로 **다시** 등장

손으로 한 번: 두 번의 반복을 따라가기

점 4개 $(1,1), (1,2), (8,8), (9,8)$, $k = 2$, 초기 $c_0 = (1,2)$, $c_1 = (8,8)$.

$$c_0 \leftarrow \left(\tfrac{1+1}{2}, \tfrac{1+2}{2}\right) = (1,\ 1.5), \qquad c_1 \leftarrow \left(\tfrac{8+9}{2}, \tfrac{8+8}{2}\right) = (8.5,\ 8)$$

2회차: 새 중심점 $(1, 1.5), (8.5, 8)$으로 다시 할당 $\rightarrow$ 네 점의 소속이 1회차와 **정확히 동일** $\Rightarrow$ **수렴** 판단, 멈춤. 최종 중심점 $(1, 1.5), (8.5, 8)$.

자주 하는 실수: 초기화의 영향을 무시

k-means는 초기화에 따라 지역 최적해에 도달

표준 대응

1. 서로 다른 무작위 초기화로 **여러 번**(예: 10회) 돌려서, 그중 **클러스터 내 분산이 가장 작은** 결과를 채택 (`sklearn.cluster.KMeans`의 기본 `n_init`이 정확히 이 역할)
2. 또는 **k-means++**처럼 서로 멀리 떨어진 점을 우선적으로 고르는 **똑똑한 초기화**로 나쁜 지역최적해에 빠질 확률 자체를 크게 줄임

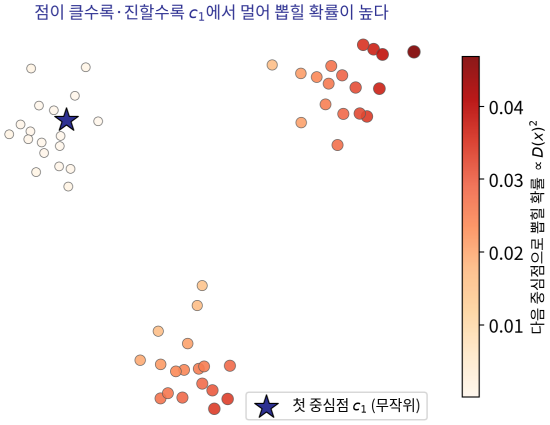
실험: 60번 중 1번이 크게 망가진다

초기화	SSE
59회	258.4 (**정답 구조**: 3개 덩어리 각각)
1번 (실패)	**1845.3** (정답의 **7.1배**)

잘못된 분할이 지역적으로 일관되어 4회차에 수렴

60번 중 1번(약 1.7%) — **데이터가 더 조밀하고 덩어리 사이 간격이 좁아질수록 이 확률은 훨씬 높아짐**. “한 번만 돌려도 대부분 맞으니까”는 **위험한 베팅**.

똑똑한 초기화: k-means++



1. 첫 중심점: 데이터에서 **균등 무작위**
2. 각 점 x에 대해 이미 고른 중심점까지의 **최소 거리** $D(x)$ 계산 $\rightarrow$ $D(x)^2$**에 비례하는 확률**로 다음 중심점 선택
3. k개 다 고를 때까지 2 반복

k-means++ vs 단순 무작위

같은 3-블롭 데이터, 초기화 20회 비교:

초기화	SSE 범위 (20회)
단순 무작위	258.4 ∼ 2210.7 (**최소한 1∼2회 크게 이탈**)
k-means++	258.4 ∼ 258.4 (**모두 동일**)

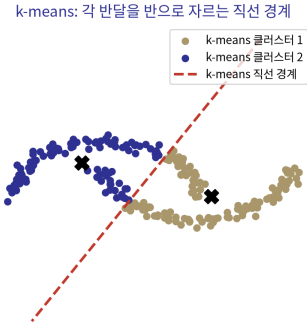
- **k-means++는 나쁜 무작위성 가능성을 줄여준다**

실전 표준 조합

k-means++ 초기화 + `n_init`회 반복 두 가지를 **함께**. `sklearn.cluster.KMeans`의 기본값이 `init="k-means++"`, `n_init=10`인 이유가 바로 이것.

k-means의 한계: “원형 덩어리” 전제





- 볼록하지 않은 구조를 못 그림
- 아웃라이어에 약함
- 모든 클러스터를 같은 크기/형상으로 취급

유명한 사례: 두 번 발견된 알고리즘

k-means의 기원 = ML 역사에서 **"동시 독립 발견"**(simultaneous discovery)의 대표 사례:

로이드 (Lloyd, 1957)

- 벨 연구소, **PCM**(상용 전화 신호를 디지털로 변환)의 양자화 문제 연구 중 발명
- 정식 논문 전에 **내부 보고서**로만 머물러 있었음

매퀸 (MacQueen, 1967)

- 스탠퍼드, **클러스터링** 문제에서 **독립적으로** 같은 알고리즘 재발명
- "k-means"라는 **이름**과 함께 발표
- 로이드가 훨씬 먼저였다는 사실이 알려지기까지 꽤 오래 걸림

두 발견은 **문제가 달랐는데**(신호 양자화 vs 데이터 클러스터링), 핵심 구조(**거리로 가장 가까운 대표에 할당** $\rightarrow$ **대표 재계산**)가 정확히 같았다.

교훈

"거리 기반 반복 재할당"은 특정 도메인의 트릭이 아니라, 거리 함수 위에서 자연스러운 최적화 구조 그 자체 — 같은 패턴이 서로 다른 문제에서 반복 등장하는 이유.

kNN과의 차이 — 그리고 단순함의 가치

kNN(게으름, lazy)

- 예측할 때마다 **그때그때** 거리 재기
- 거리 계산이 **예측 시점**에 몰려 있음

k-means

- 중심점이 **수렴할 때까지** 데이터를 반복적으로 훑음
- 거리 계산이 "**학습** 단계"에 몰려 있음 ⇒ 오히려 **선형회귀 같은 파라미터 학습** 쪽에 더 가깝다
- **라벨이 없다는 것만** 지도학습과 다름

단순함이 곧 약점이 아니다

kNN과 k-means 둘 다 모델이 "무엇을 학습했는지"를 설명해주는 복잡한 이론이 없지만, **바로 그 단순함**(순전히 거리 계산) 덕분에 **언제, 왜 실패하는지도 수식으로 정확히** 짚어낼 수 있다.

자주 묻는 질문 (4.3)

- **Q. k-means가 항상 수렴하나요?** 그렇다 — 매 반복마다 SSE가 절대 늘지 않고 0 이상이므로 **유한한 시간 안에** 수렴이 보장. 다만 **어디로** 수렴하는지(전역 vs 나쁜 지역최적해)는 보장되지 않음 = "자주 하는 실수"의 핵심
- **Q. k를 몰라도 쓸 수 있나요?** k-means 자체는 k를 **미리** 정해야 함. k를 데이터로부터 자동으로 원하면: 엘보우로 여러 k를 비교하거나, 4.4절 EM/GMM처럼 각 클러스터의 "적합도"를 **확률적으로** 비교할 수 있는 모델로 넘어가기
- **Q. 왜 항상 "원형" 덩어리만 찾나요? 구조적으로** 결정됨 — 두 클러스터의 경계는 "두 중심점까지 거리가 같은 점들의 집합"이고, 2차원에선 **직선**(수직이등분선) $\Rightarrow$ 각 클러스터 영역은 반드시 **볼록** 다각형(보로노이 셀). 반달·고리 모양 경계를 그릴 **자유 자체가 모델 안에 없음** — 파라미터 튜닝으로 극복할 수 없는 **구조적** 한계
- **Q. 클러스터가 비어(멤버 0)질 수 있나요?** 이론적으로 가능 — 어떤 중심점도 어떤 점에게도 "가장 가까운"이 아닌 상황. `if cluster else centroids[i]` 분기가 바로 이걸 막음(빈 클러스터의 "평균"은 정의되지 않음 $\rightarrow$ 중심점 유지가 관례)
- **Q. 차원이 높으면 k-means도 안 쓰나요?** 4.2절의 차원의 저주가 **그대로** 적용 — "가장 가까운 중심점" 구분이 점점 무뎌짐. 실무 표준: **Week 14 PCA로 차원 축소**한 뒤 2∼3차원 공간에서 클러스터링

4.4 GMM, EM 알고리즘, 그리고 LDA

닭이 먼저냐 달걀이 먼저냐: 잠재변수 문제

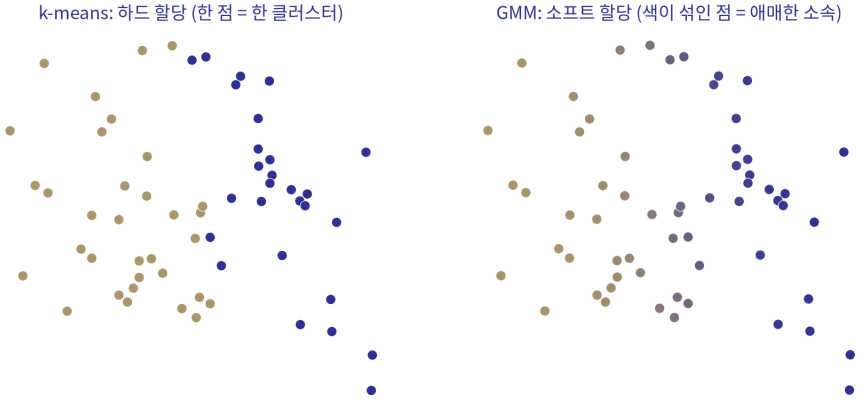
- **GMM: 정규분포 + 소프트 할당**
- 문제는 각 점의 **진짜** 클러스터 z가 데이터에 **관측되지 않음**(**잠재변수**, latent variable)

순환

- z(진짜 클러스터)를 **안다면** $\rightarrow$ 각 클러스터의 평균·분산 구하기 **쉬움**(GDA처럼)
- 평균·분산을 **안다면** $\rightarrow$ 각 점의 **사후확률** z 구하기 **쉬움**(베이즈 정리)
- **둘 다 모른다** = 닭 vs 달걀의 순환

EM: 반복적 고정-추정

그림으로: 하드 할당 vs 소프트 할당



같은 두 덩어리, 왼쪽은 k-means(모든 점이 둘 중 하나의 색), 오른쪽은 GMM(경계 근처 점들이 두 색이 섞인 회색조 → “60%는 파랑, 40%는 갈색”의 확률적 소속).

유명한 사례: 절차의 이름이 곧 알고리즘 이름

- **EM은 별개 문제를 하나의 구조로 통합**
- “**EM**”이라는 이름 자체가 E-step과 M-step이라는 **두 절차의 앞글자** 그대로 = 이 논문의 성격



Arthur P. Dempster (1929–2026) — Harvard 통계학과
Dempster, Laird & Rubin(1977) “Maximum Likelihood from Incomplete Data via the EM Algorithm”의 제1저자

GMM의 생성 과정

1. **라벨 뽑기**: K개 면의 **주사위**(카테고리 분포)를 굴려 $z_i \in \{1, \ldots, K\}$ 뽑기 — 각 면의 확률 π_k
2. **점 뽑기**: 뽑힌 클러스터 $z_i = k$의 정규분포 $\mathcal{N}(\mu_k, \Sigma_k)$에서 점 x_i 샘플링

(1차원 예제에선 Σ_k가 스칼라 σ_k^2로 쓰인다.) **모수 세 종류**: 클러스터별 평균 μ_k, 분산 σ_k^2, **혼합비율** π_k ($\sum_k \pi_k = 1$).

이 모델이 정의하는 한 점의 likelihood:

$$p(x) = \sum_{k=1}^{K} \pi_k \, \mathcal{N}(x; \mu_k, \sigma_k^2)$$

왜 MLE로 바로 풀리지 않는가: 곱의 합

Week 3 GDA / 나이브베이즈

- "**라벨이 주어지면**" 파라미터별 likelihood가 **곱의 합**으로 깔끔하게 풀림
- ⇒ **닫힌 형태**의 MLE 공식 존재

GMM

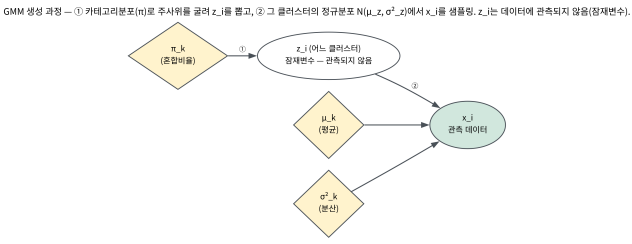
- 라벨 z가 **관측되지 않아**
- 전체 likelihood = $\prod_i \left(\sum_k \pi_k \mathcal{N}(x_i; \mu_k, \sigma_k^2) \right)$
- = **곱의 합**

합이 곱 안에 끼어 있으면 미분을 0으로 놓고 풀 수 있는 구조가 아니다.

EM의 본질

"만약 z가 **알렸다면**"은 Week 3 GDA와 **똑같이** 닫힌 형태로 풀린다 — EM은 이 **"만약"을 반복적으로 실행**하는 알고리즘.

잠재변수 다이어그램



- **마름모**$(\pi_k, \mu_k, \sigma_k^2)$ = **모수**
- **빈 원** z_k = **관측되지 않는 잠재변수**(어느 클러스터인지)
- **채워진 원** x_i = **관측 데이터**
- π_k가 z_k를, z_k와 (μ_k, σ_k^2)가 x_i를 **만든다**

"관측되지 않은 구조(z)가 관측된 데이터(x)를 만든다" — Week 15 생성 모델 전체가 이 그림의 변주.

EM 알고리즘: E-step

E-step: 사후확률 계산

$$\gamma_{ik} = P(z_i{=}k \mid x_i) = \frac{\pi_k \mathcal{N}(x_i; \mu_k, \sigma_k^2)}{\sum_j \pi_j \mathcal{N}(x_i; \mu_j, \sigma_j^2)}$$

- $\sum_k \gamma_{ik} = 1$ (각 점은 모든 클러스터에 확률 합 1로 분배됨)

EM 알고리즘: M-step

M-step (Maximization) — 방금 구한 책임값을 **"가중치"**로 삼아, 각 클러스터의 파라미터를 **다시 추정** — γ_{ik}가 클수록 그 점이 μ_k, σ_k^2 추정에 더 크게 기여:

$$\mu_k = \frac{\sum_i \gamma_{ik} x_i}{\sum_i \gamma_{ik}}, \qquad \sigma_k^2 = \frac{\sum_i \gamma_{ik}(x_i - \mu_k)^2}{\sum_i \gamma_{ik}}, \qquad \pi_k = \frac{\sum_i \gamma_{ik}}{m}$$

한 번 더 천천히 보면 — 정말 새 수식이 아님

"라벨이 **전부** k인 점들의 **평균·분산·비율**"(Week 3의 **닫힌 형태 MLE**)에서, 각 점의 "소속 여부"(0 또는 1)를 **책임값** γ_{ik}(0과 1 사이의 실수)로 **바꾼 것**뿐. E-step = "**하드 라벨 대신 소프트 라벨을 채워넣는**" 단계, M-step = "**소프트 라벨이 주어졌을 때의 닫힌 형태 MLE**" — **EM의 전부가 이 한 문장**.

em_step 코드 (1차원)

```python
import math

def gaussian_pdf(x, mu, sigma):
    return (1 / (sigma * math.sqrt(2 * math.pi))) * \
           math.exp(-((x - mu) ** 2) / (2 * sigma ** 2))

def em_step(data, means, sigmas, weights):
    K, n = len(means), len(data)
    # E-step: 책임값(responsibility) 계산
    resp = []
    for x in data:
        probs = [weights[k] * gaussian_pdf(x, means[k], sigmas[k])
                 for k in range(K)]
        total = sum(probs)
        resp.append([p / total for p in probs])
    # M-step: 책임값으로가중평균한파라미터재추정
    Nk = [sum(resp[i][k] for i in range(n)) for k in range(K)]
    new_means = [sum(resp[i][k] * data[i] for i in range(n)) / Nk[k]
                 for k in range(K)]
    new_sigmas = [math.sqrt(sum(resp[i][k] * (data[i] - new_means[k]) ** 2
                                for i in range(n)) / Nk[k])
                  for k in range(K)]
    new_weights = [Nk[k] / n for k in range(K)]
    return new_means, new_sigmas, new_weights
```

왜 우도가 항상 증가하는가: 엔센 부등식 (1)

하나의 점 x에 대해, γ를 z 위의 **아무** 확률분포라고 놓고:

$$p(x) = \sum_k p(x, z{=}k) = \sum_k \gamma_k \frac{p(x, z{=}k)}{\gamma_k} = \mathbb{E}_{z\sim\gamma}\left[\frac{p(x,z)}{\gamma(z)}\right]$$

log**은 오목함수**(concave) $\Rightarrow$ **엔센 부등식** $\log \mathbb{E}[X] \geq \mathbb{E}[\log X]$ 적용:

왜 우도가 항상 증가하는가: 엔센 부등식 (2)

$$\log p(x) \geq \mathbb{E}_{z\sim\gamma}[\log p(x,z)] - \mathbb{E}_{z\sim\gamma}[\log \gamma(z)] \quad \stackrel{\text{def}}{=} \quad \mathcal{F}(\theta,\gamma)$$

$$\log p(x;\theta_{\text{new}}) \geq \mathcal{F}(\theta_{\text{new}},\gamma) \geq \mathcal{F}(\theta_{\text{old}},\gamma) = \log p(x;\theta_{\text{old}})$$

우도가 매 반복마다 절대 줄지 않는다.

6개 데이터로 확인: 단조증가는 성립

이 간극이 정확히 KL 발산이다

- 우도와 하한 $\mathcal{F}$의 간극을 정확히 쓰면:

$$\log p(x;\theta) - \mathcal{F}(\theta,\gamma) = \sum_k \gamma_k \log \frac{\gamma_k}{p(z{=}k|x)} = D_{KL}\big(\gamma \,\|\, p(z|x)\big)$$

Week 15.1 ELBO와 같은 식

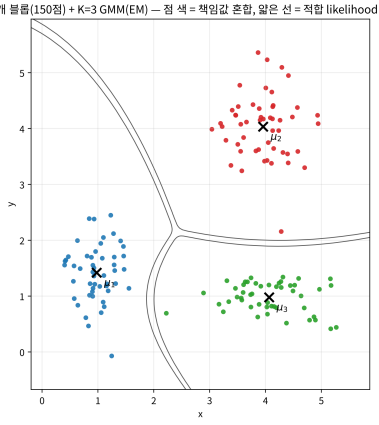
$D_{KL}(q_\phi(z|x) \,\|\, p(z))$는 **표기만 다를 뿐** 지금 이 $D_{KL}(\gamma \| p(z|x))$와 **같은 자리**에 있는 항 — z가 **이산이고 사후확률을 정확히 구할 수 있으면(GMM)** 이 KL을 0으로 만들 수 있고, z가 **연속이라 정확히 못 구하면(VAE)** 신경망으로 이 KL을 **최대한 줄이는** 것이 최선이 된다.

2차원 GMM: 공분산 행렬 Σ_k

1차원에선 μ_k, σ_k^2가 스칼라였지만 d차원에선 $\mu_k \in \mathbb{R}^d$, $\Sigma_k \in \mathbb{R}^{d\times d}$(**공분산 행렬**). E-step은 **그대로**, M-step은 **가중 공분산**만 추가:

$$\Sigma_k = \frac{\sum_i \gamma_{ik}(x_i-\mu_k)(x_i-\mu_k)^\top}{\sum_i \gamma_{ik}}$$

이 Σ_k가 **GMM이 k-means보다 표현력이 많은 정체** — k-means는 **위치**(μ_k) 하나뿐(경계 **항상 직선**, "동일한 크기의 원")인데, Σ_k는 (i) **크기**, (ii) **타원형 모양**, (iii) **기울어진(비대각) 형상**을 **모두** 표현. 결정경계 도 "두 정규분포의 **밀도비가 1인 곡선**" ⇒ 직선이 아닌 **타원호**.

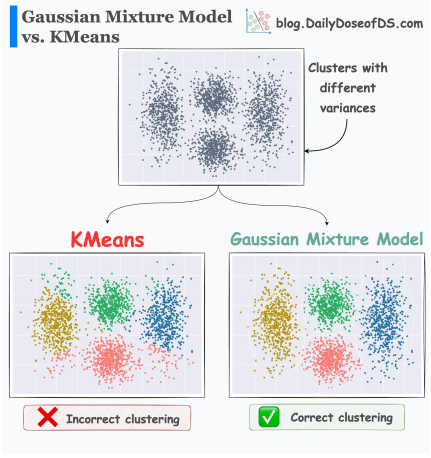


3개 블롭(각 50점)에 $K=3$ GMM을 EM으로 학습. 점 색=**책임값**, 등고선=likelihood. 왼쪽은 **길쭉한 타원**, 위쪽은 **원**.

k-means와의 가장 눈에 띄는 차이

덩어리 **경계 안쪽**의 점들이 **두 클러스터 색을 섞어서** 보임 — k-means(점 하나에 딱 하나의 색)와 달리 **불확실성을 그대로** 보여준다.

이 차이를 한눈에: 분산이 다른 클러스터



k-means는 동일한 크기의 원만 그릴 수 있어

손으로 한 번: EM 한 라운드 (E + M)

점 3개 $x = 0, 3, 8$, $K = 2$, 초기 $\mu_1 = 1, \mu_2 = 5$, $\sigma_1 = \sigma_2 = 1$, $\pi_1 = \pi_2 = 0.5$.

E-step — 각 점의 두 밀도와 책임값:

점	두 밀도
0	$\mathcal{N}(0;1,1) = 0.242$, $\mathcal{N}(0;5,1) \approx 10^{-5}$
3	$\mathcal{N}(3;1,1) = \mathcal{N}(3;5,1) = 0.054$ (**정중앙**이라 두 밀도 같음)
8	$\mathcal{N}(8;1,1) \approx 0$, $\mathcal{N}(8;5,1) = 0.00443$

M-step ($N_1 = N_2 = 1.5$):

$$\mu_1 = \frac{1 \cdot 0 + 0.5 \cdot 3 + 0 \cdot 8}{1.5} = 1.0, \qquad \mu_2 = \frac{0 + 0.5 \cdot 3 + 1 \cdot 8}{1.5} = 6.33$$

$$\sigma_1^2 = \frac{1(0-1.0)^2 + 0.5(3-1.0)^2}{1.5} = 2.0 \ (\sigma_1{=}1.41), \qquad \sigma_2^2 = \frac{0.5(3-6.33)^2 + 1(8-6.33)^2}{1.5} = 5.56 \ (\text{}$$

K를 고르는 법: BIC

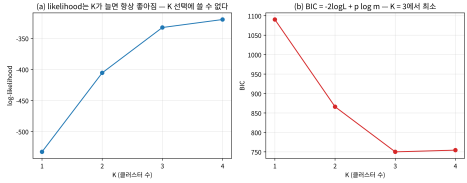
대신 **BIC**(Bayesian Information Criterion):

$$\text{BIC} = -2 \log \mathcal{L}(\hat{\theta}) + p \log m$$

BIC: likelihood vs 벌점 트레이드오프

3-블롭 150점에 $K = 1 \sim 4$ EM (BIC 최소 = **정확히** $K = 3$):

K	log-likelihood	BIC
1	-749.1	1513.2
2	-630.6	1296.2
3	-410.1	**875.4**
4	-410.1	895.4



같은 EM, 다른 데이터: LDA 개요

GMM을 생성 과정 관점에서 보면, "데이터가 **어디서** 만들어졌는가"(잠재변수 z_i 한 줄기)를 모델링. 그런데 데이터가 **점**으로만 만들어지는 것은 아니다: **문서**(뉴스·논문·리뷰)의 경우 "이 문서가 **어떤 주제로** 만들어졌는가"를 물으면, 잠재변수는 K**개 주제의 혼합 비율** $\theta_d (\sum_k \theta_{dk} = 1)$가 된다.

이 질문을 **같은 EM의 틀**로 푸는 것이 **LDA**(Latent Dirichlet Allocation, 잠재 디리클레 할당) — 2003년 **블레이(Blei)·응(Ng)·조던(Jordan)**.

- 이름의 **Dirichlet** = 사전분포의 이름, **Allocation** = "할당" — **문서가 토픽에 할당되고, 토픽이 단어에 할당**된다는 생성 과정
- "**GMM의 "점 → 클러스터"를 "단어 → 토픽"으로 바꾼 것**" — 같은 EM 계열의 고전 확률모델
- Week 3 나이브베이즈의 단어 **로그 우도비**는 사전분포 없이 "스팸스러움"만 쟀다 — LDA는 거기에 **사전분포**를 더해 "이 문서가 어떤 주제들의 혼합인가"까지 완전한 베이즈로 추정한다.



Andrew Ng — Blei, Ng & Jordan(2003)
“Latent Dirichlet Allocation”(LDA) 공동저자, JMLR 3, 993–1022

LDA의 생성 과정: GMM 골격 그대로

1. **토픽 혼합 θ_d는 Dir(α)에서 뽑음**
2. **단어 w_{dn}은 토픽 z_{dn}에서 뽑음**
   - **단어 프로필 ϕ_k는 Dir(β)에서 뽑힘**

켤레성의 선물

디리클레 분포는 단순히 “확률벡터 위의 분포”가 아니라 **다항분포의 켤레(conjugate) 분포**라 **E-step과 M-step이 전부 닫힌 형태로 유지** — GMM의 정규-감마 구조가 해준 것과 **정확히 같은 역할**.

LDA의 E/M-step: GMM과의 대비

- **E-step: 단어 위치의 토픽 사후확률**
- **M-step: 책임값으로 파라미터 갱신**

GMM vs LDA 한 표로

	GMM	**LDA**
관측 데이터	점 $x_i \in \mathbb{R}^d$	문서의 **단어** w_{dn}(이산)
잠재변수	클러스터 라벨 z_i(1개/점)	토픽 z_{dn}(1개/단어) + 혼합 θ_d(1개/문서)
“덩어리”의 모양	정규분포 $\mathcal{N}(\mu_k, \Sigma_k)$	**단어 분포** ϕ_k(디리클레 사전 위)
“클러스터”의 의미	특징공간에서 **가까이 모인 점**	**자주 함께 나타나는 단어 패턴**(토픽)
M-step 갱신	μ_k, Σ_k, π_k	θ_d, ϕ_k

한 줄

“클러스터”가 특징공간에서의 **기하학적 근접성**(GMM)이 아니라 **단어 공출현의 통계적 패턴**(LDA)으로 바뀜 — 이것이 **같은 EM**의 두 가지 얼굴을 가질 수 있는 이유. K 선택 문제도 그대로 이어짐(토픽 수 선택도 **BIC 계열 트레이드오프** — 다만 우도를 닫힌 형태로 계산할 수 없어 **변분 하한**을 기준으로 삼음).

자주 하는 실수 3가지

1. **"likelihood가 가장 큰 K를 고른다"** — $K = m$이 **항상** 이김(위 BIC 섹션). **BIC/AIC/교차검증**을 써야

자주 묻는 질문 (4.4)

- **Q. EM이 항상 전역 최적해로 수렴하나요? 아니** — 매 반복마다 우도가 절대 **감소하지 않음**(그래서 언젠가 수렴)은 보장되지만, 4.3절 k-means와 마찬가지로 초기 파라미터에 따라 **지역 최적해**에 갇힐 수 있음. 실전: **서로 다른 초기화로 여러 번 돌려 가장 좋은 것을 채택**(k-means와 정확히 같은 대응책)
- **Q. GMM이 k-means보다 항상 더 좋은가? 아니** — GMM은 각 클러스터가 정규분포를 따른다는 가정(Week 3.2 GDA와 동일) + 부드러운 soft 할당 계산 ⇒ k-means보다 **계산 비용 더 듦**. 클러스터가 **겹쳐 있거나 모양이 서로 다를** 가능성이 있으면 GMM 유리, **뚜렷이 분리돼 있고 속도가 중요**하면 k-means로 충분
- **Q. GMM의 E-step이 "정확한" 사후확률을 계산 한다면, VAE는 왜 신경망으로 근사하나요?** GMM의 잠재변수 z는 **이산**이고 가능치가 K개 뿐이라, 사후확률을 K개 항의 합으로 **정확히** 계산 가능. VAE(15.1절)의 잠재변수는 **연속** 벡터라 가능한 z가 **무한**하고, 사후확률의 **적분이 계산 불가능**(intractable) ⇒ "계산 안 되는 E-step"을 **신경망** $q_\phi(z|x)$로 근사하고 M-step도 **역전파**로 — VAE는 정확히 이 필요에서 나온 **"EM의 약한 근사 버전"**
- **Q. EM은 얼마나 빨리 수렴하나요?** 통상 **선형(linear) 수렴** — 수렴점 가까울수록 오차가 "고정된 비율"로 줄어들어 **마지막 몇 자리 정밀도는 의외로 오래** 걸림. 실전: "파라미터 변화가 ϵ 미만"이나 "likelihood 변화가 ϵ 미만" 같은 **수렴 기준**. 고차원에선 Σ_k가 **특이**(singular, 역행렬이 없는) 행렬에 **가깝게** 수렴하기 쉬우니, 실전 라이브러리 (`sklearn.mixture.GaussianMixture` 등)는 (i) **대각 공분산**만 허용, (ii) 모든 클러스터 **같은 공분산**, (iii) $\Sigma_k + \epsilon I$ 같은 **정규화** 옵션 제공
- **Q. "생성 모델"이라면서 어떻게 데이터를 만들어냅니까?** 학습이 끝나면 **완전한** 데이터 생성 절차 $p(z) \to p(x|z)$를 가짐 — 새 점: (1) 주사위 π로 z 뽑고, (2) 그 $z = k$의 $\mathcal{N}(\mu_k, \Sigma_k)$ 에서 x 샘플링. 이 "생성" 능력이 Week 15(VAE가 $z \sim \mathcal{N}(0, I)$에서, GAN이 **노이즈** 에서)의 "데이터 생성"과 **정확히 같은 질문**의 가장 단순한 버전

실습으로 연결하기 (4.4)

노트북 `chapter04_4_gmm_em.ipynb`은 numpy/matplotlib만으로 본문 코드를 그대로 실행:

1. 책의 `em_step`을 **6개 데이터**에 돌려 **수렴 과정 추적**(1차원) — 앞의 $-15.56 \to -6.05 \to -6.05$ 표
2. 3개 블롭 2차원 데이터에서 **2D GMM**을 EM으로 학습, **등고선 + 책임값 기반 색칠**로 시각화
3. **무작위 초기화 30회** 돌려 **지역 최적해/분산 붕괴 실패 사례**를 실제로 만든 뒤 좋은 초기화와 log-likelihood로 비교
4. $K = 1 \sim 4$의 **BIC 곡선**을 그려 K 선택 확인(최소 $K = 3$)

본문의 특정 숫자(실패 횟수, BIC 값 등)는 재현을 목표로 하지는 않으며, 같은 **"현상"**이 실제로 일어나는지 확인하는 것.

이번 주차 정리

1. **4.1 kNN** — 게으른·비모수 방법, 예측 비용 $O(m \cdot d)$. **정규화 빼먹으면** 0.556→1.000 반전의 실수. k를 키우며 **편향-분산** 트레이드오프: 톱니바퀴(과적합) → 직선(과소적합). $k = 1$ 결정경계 = **보로노이 다이어그램**.
2. **4.2 차원의 저주** — 부피 0.5^n 증발, $p^{1/n}$ 스케일, **이웃이 동거리에** $(11^{1/n} \to 1)$. junk 축 50개 → 정확도 0.867→**0.507**. k는 **검증셋**으로 고르고, "훈련 정확도로 고르면 **항상** $k = 1$" 함정.
3. **4.3 k-means** — "할당→갱신"이 **SSE를 단조감소**시켜 수렴(단, **어디로**는 보장 안 됨). **엘보우**로 k 선택, **k-means++** ($D(x)^2$-비례, $O(\log k)$ 보장) + `n_init`. 경계가 **항상 직선(볼록)**이라는 **구조적 한계**. 로이드(1957)·매퀸(1967) 동시 발견.
4. **4.4 GMM/EM** — 하드 → **소프트 할당**. 라벨·파라미터 "둘 다 모른다"는 순환을 E-step(책임값) → M-step(가중 평균)으로 반복. **옌센 부등식 4줄** = "우도가 절대 줄지 않는다". Σ_k가 **타원형·기울어진** 클러스터. K는 **BIC**로, **분산 붕괴** 퇴화 해 주의. **LDA** = GMM의 "점→클러스터"를 "단어→토픽"으로.

다음 주차 — Week 5. SVM과 커널

kNN의 결정경계는 "이웃 순위"에 따라 결정되는 **국소적** 경계(보로노이). 다음 장은 반대로 "무수히 많은 경계선 중 **어떤 것이 새 데이터에 가장 안전한가**"라는 **전역적·기하학적** 질문을 던지며, 그 답으로 **마진 최대화(SVM)**(Cortes & Vapnik 1995) 와 **커널 트릭**을 소개한다.